香港城市大學
City University of Hong Kong
Innovating into the Future

Tutorial 3: Apollo Cyber -- Component

Component in Cyber

- **Node:** The basic functional unit and computational process within Cyber
- Every independent algorithm module, driver, or service exists as one or more nodes.
  - Serves as the container for component logic and lifecycle management
  - All functional code executes within a Node, which acts as the sole entry point to the Cyber communication network.
  - Nodes adhere to the "high cohesion, loose coupling" principle. They can be developed, compiled, deployed, started, and stopped independently, significantly enhancing system maintainability and scalability.
- **Channel:** The bus or pipeline for **asynchronous data communication** between components in Cyber
  - Serves as the transmission medium for messages (e.g., sensor data, control commands, status information). Each Channel has a unique name and a strictly defined data type.

Component in Cyber

- **Writer & Reader:** Tool classes attached to a Node that handle the actual reading and writing of messages.
  - **Writer:** Created by a Node that publishes messages. It is responsible for writing data to a specified **Channel**.
  - **Reader:** Created by a Node that receives messages. It is responsible for reading data from a specified **Channel**.
  - **Relationship:** A single Channel can host multiple Writers and multiple Readers simultaneously, enabling flexible data sharing. The Reader/Writer mechanism ensures encapsulated communication and type safety.

Component in Cyber

- **DAG (Directed Acyclic Graph):** A model used to describe the topological relationships of computational tasks within the entire system.
  - It provides a graphical representation that clearly defines:

    What the **Nodes** are.

    How data flows (via **Channels**) between Nodes (through their **Readers/Writers**).

    The dependencies and execution order among Nodes.
  - **Value:**

    Visualization: Makes complex system data flows immediately understandable. It is a crucial tool for comprehending, debugging, and optimizing system architecture.

    Scheduling Optimization: Provides the foundation for the CYBERRT runtime scheduler, enabling it to efficiently manage Node execution and resource allocation to optimize overall performance.

Component in Cyber

Types of Components

- **Data-Driven Component:** Supports single or multiple data input channels. When all data required by the Proc function is ready, the Scheduler packages the Proc function and the data for execution in the Thread Pool.
- **Timer-Driven TimerComponent:** Supports timers with a minimum precision of 2 ms, driven by a time wheel mechanism. The Proc function is executed at fixed intervals.

Data Fusion in Component

- Components provide a simple message fusion mechanism
- Support up to 4-channel message fusion
- When reading data from multiple Channels, the first Channel is designated as the primary Channel. Whenever a new message arrives on the primary Channel, Cyber will call the Component's Proc() function to perform one round of data processing.

```
template <typename M0, typename M1, typename M2, typename M3>
bool Component<M0, M1, M2, M3>::Process(const std::shared_ptr<M0>& msg0,
                                        const std::shared_ptr<M1>& msg1,
                                        const std::shared_ptr<M2>& msg2,
                                        const std::shared_ptr<M3>& msg3) {
  if (is_shutdown_.load()) {
    return true;
  }
  return Proc(msg0, msg1, msg2, msg3);
}
```

TimerComponent

- TimerComponent does not provide data fusion

```
bool TimerComponent::Initialize(const TimerComponentConfig& config) {
  if (!config.has_name() || !config.has_interval()) {
    AERROR << "Missing required field in config file.";
    return false;
  }
  node_.reset(new Node(config.name()));
  LoadConfigFiles(config);
  if (!Init()) {
    return false;
  }
  interval_ = config.interval();

  std::shared_ptr<TimerComponent> self =
      std::dynamic_pointer_cast<TimerComponent>(shared_from_this());
  auto func = [self]() { self->Process(); };
  timer_.reset(new Timer(config.interval(), func, false));
  timer_->Start();
  return true;
}
```

Read the time interval from the component configuration, and reset and start a timer in the initialization function.

Component Creation

- Include Component header files
  - Choose the type of component as needed
- Define a class inheriting the component
- Rewrite Init() and Proc()
  - Init(): initialization, executed when the component is loaded
  - Proc(): the core function implementing the functionality of the component
- Register the component in Cyber

ComponentSample.h

```cpp
//1、包含头文件
#include "cyber/component/component.h"
#include "test/proto/examples.pb.h"

// 2、定义一个类，继承以Driver和Chatter为参数的Component模版类
class ComponentSample : public Component<Driver, Chatter> {
 public:
 //3、重写Init() 函数和Proc() 函数
  bool Init() override;
  bool Proc(const std::shared_ptr<Driver>& msg0,
            const std::shared_ptr<Chatter>& msg1) override;
};
//4、 在cyber中注册ComopnentSample
CYBER_REGISTER_COMPONENT(ComponentSample)
```

Advantages of using Component

- Modular Architecture & Process Isolation
  - Launch Files Manage Independent Processes

    Each component runs as a separate process, ensuring fault isolation and system stability.

    Enables distributed deployment across multiple compute nodes for scalability.
- Configuration Without Recompilation
  - DAG Files Define System Topology

    Modify component connections, channels, and parameters at runtime via simple configuration files.

    Eliminates downtime from recompilation, accelerating development and debugging cycles.
- Flexible Communication & Fusion Strategies
  - Multi-Channel Input with Smart Fusion

    Supports up to 4 data channels with configurable fusion logic (time-based or primary-triggered).

    Handles diverse message types (sensor data, control commands, state updates) seamlessly.
- Simplified Development & Runtime Agility
  - Minimalist Interfaces, Maximum Flexibility

    Easy-to-implement APIs reduce boilerplate code and accelerate component creation.

    Dynamic Loading: Components can be started, stopped, or swapped at runtime by the Cyber framework.